Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets

A semantic, accessible, well-laid-out page still fails users if the main image takes four seconds, content jumps while loading, or every click freezes for half a second — and fails its owners if search engines can't understand it. This chapter covers **web performance** (critical rendering path, render-blocking resources, Core Web Vitals — measured on real pages in headless Chrome) and **SEO basics**, then turns the book into revision material: 47 interview questions linked to the chapters, one-page cheat sheets, and a glossary of more than 200 terms.

In this chapter: 14.1 The critical rendering path · 14.2 Render-blocking resources (measured: FCP 848 → 40 ms) · 14.3 Core Web Vitals (LCP, INP, CLS, measured with `PerformanceObserver`) · 14.4 Fixing each metric · 14.5 SEO basics (title, meta, canonical, JSON-LD) · 14.6 Interview questions · 14.7 Cheat sheets · 14.8 Glossary.

How the numbers were made. Performance only means something on a slow network, so `code/serve.mjs` fakes one without a server: headless Chrome requests `http://site.test/<file>`, and Puppeteer intercepts each request and answers from `code/` after an artificial delay ("the stylesheet takes 800 ms"). Absolute timings are machine-dependent but repeat within a few ms. Metrics are read with the browser's performance APIs (`PerformanceObserver` delivers timing records) — the same ones the `web-vitals` library and **real-user monitoring** (RUM: timings collected from actual visitors) use. Thresholds were checked on web.dev and SEO rules on Google Search Central (Google's documentation for site owners) in September 2026.

14.1 The critical rendering path

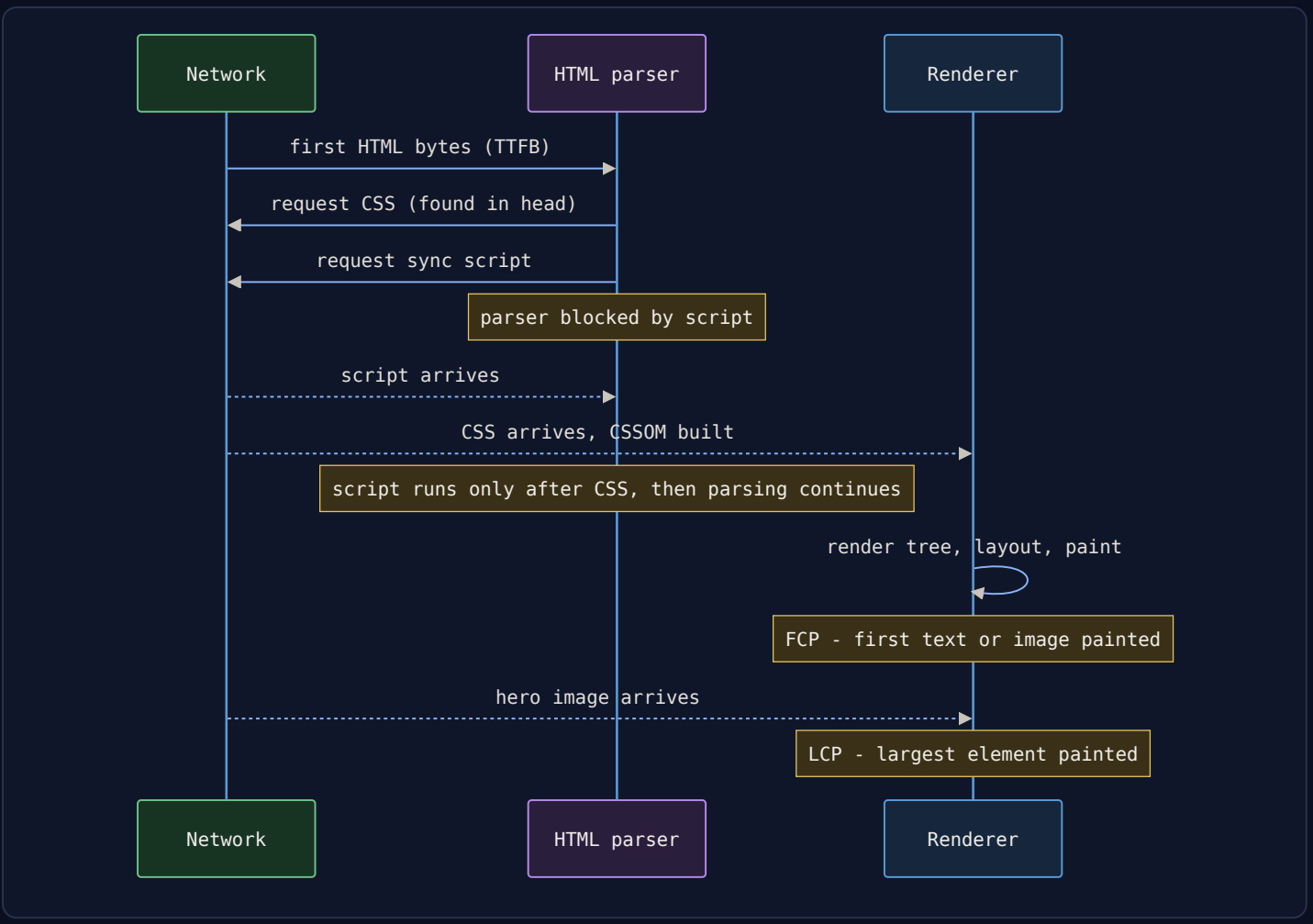
What it is

The **critical rendering path** (CRP) is the minimum work before the first pixels: HTML → **DOM**, CSS → **CSSOM**, both → **render tree** (displayed elements with final styles) → **layout** → **paint** → **composite** (§1.7 walks through each). Here we care about **what makes it slow**.

A **resource** (stylesheet, script, image, font) is **critical** if the first render must wait for it. By default:

- **CSS is render-blocking**: no paint until every `<head>` stylesheet is loaded and parsed — otherwise unstyled content would show and then jump (**FOUC**, flash of unstyled content, plus a reflow).
- **A classic `<script>` without `async` / `defer` is parser-blocking**: the HTML parser stops, downloads the script, *waits for pending stylesheets* (the script might read computed styles), runs it, then continues; nothing below it is in the DOM yet. (Render-blocking delays painting; parser-blocking stops the DOM being built.)
- **Images, fonts, `async` / `defer` scripts aren't render-blocking** — but can still delay what users feel (main image, text appearing).

How it works — the timeline and its metrics



Metric	Measures	Good (75th percentile — the value 75 % of real visits beat or match)
TTFB (Time To First Byte)	Server + network until the first HTML byte	≤ 0.8 s
FCP (First Contentful Paint)	First text/image painted	≤ 1.8 s
LCP (Largest Contentful Paint)	Main content painted (Core Web Vital, 14.3)	≤ 2.5 s
INP, **CLS**	Responsiveness and visual stability (Core Web Vitals)	≤ 200 ms, ≤ 0.1

The three levers

1. **Fewer critical resources** — inline small critical CSS, `defer` scripts, don't load what the first screen doesn't need.
2. **Fewer critical bytes** — minify and compress (gzip/Brotli/zstd), remove unused CSS/JS.
3. **Shorter critical path length** (round trips — request, wait, then the next) — make critical resources discoverable in the HTML, preconnect to other **origins** (scheme + host + port), avoid chains (CSS `@import`ing CSS, JS injecting the main image).

14.2 Render-blocking resources

The experiment

One article page with three `<head>`s; the stylesheet takes **800 ms**, the script **600 ms** (code/01_render_blocking.mjs).

code/01_blocking.html — the classic mistake:

```
<link rel="stylesheet" href="01_styles.css">   <!-- render-blocking -->
<script src="01_app.js"></script>              <!-- parser-blocking -->
```

code/01_defer.html — only the script fixed:

```
<link rel="stylesheet" href="01_styles.css">
<script src="01_app.js" defer></script>
```

code/01_optimized.html — critical CSS inlined, the rest loaded without blocking:

```
<style>/* only what the first screen needs */ body{…} header{…} main{…}</style>
<link rel="preload" href="01_styles.css" as="style" onload="this.onload = null; this.rel = 'stylesheet'">
<noscript><link rel="stylesheet" href="01_styles.css"></noscript>
<script src="01_app.js" defer></script>
```

Output:

```
01_blocking.html   FCP  848 ms   DOMContentLoaded  833 ms   load  834 ms   app.js ran at 833 ms
                   01_app.js blocking (done at 632 ms)  |  01_styles.css blocking (done at 833 ms)
01_defer.html      FCP  844 ms   DOMContentLoaded  833 ms   load  837 ms   app.js ran at 833 ms
                   01_app.js non-blocking (done at 631 ms)  |  01_styles.css blocking (done at 832 ms)
01_optimized.html  FCP   40 ms   DOMContentLoaded  631 ms   load  631 ms   app.js ran at 631 ms
                   01_app.js non-blocking (done at 630 ms)  |  01_styles.css non-blocking (done at 830 ms)
```

(`blocking` / `non-blocking` is Chrome's own classification from `PerformanceResourceTiming.renderBlockingStatus` .)

1. **Blocking**: nothing for ~850 ms; the script downloaded by 632 ms but **ran at 833 ms** — it waited for the stylesheet.
2. **Only** `defer` : FCP unchanged (848 → 844 ms) because the stylesheet was the bottleneck. Fixing the wrong resource buys nothing — find the *critical* one first. (`defer` still helped: the parser never stopped.)
3. **Optimized**: first paint at **40 ms**, ~20× sooner. The preloaded stylesheet arrived at 830 ms and applied itself (`onload` flips `rel` to `stylesheet` without ever blocking render); `load` fired at 631 ms without waiting for it. (`DOMContentLoaded` = HTML parsed and deferred scripts run; `load` = page and blocking resources loaded.)

The toolbox

Resource	Default	Better
Stylesheet	Render-blocking	Keep small; inline critical CSS (≈ first screen, a few KB); load the rest non-blocking; a non-matching `media` (e.g. `print`) makes it non-blocking (still downloaded, low priority); split per page
Classic script in `<head>`	Parser-blocking	`defer` (in order after parsing, before `DOMContentLoaded`) or `async` (on arrival, any order — independent scripts like analytics); `type="module"` is deferred by default
Web font	Text invisible until loaded (up to ~3 s)	`font-display: swap` / `optional`, `<link rel="preload" as="font" type="font/woff2" crossorigin>`, subset, self-host (§11.2)
Main image	Discovered when the parser reaches it	In the HTML, `fetchpriority="high"`, never `loading="lazy"` above the fold (§4.4)
Other origins (a CDN — content delivery network of nearby copy servers — font hosts)	DNS + TCP + TLS on first use — each at least one round trip (§1.3–1.4)	`<link rel="preconnect" href="https://cdn.example">` for the 1–3 most important
`@import` in CSS	Serial chain: CSS → discover → fetch CSS	`<link>` both files, or bundle

```
<!-- defer vs async -->
<script src="app.js" defer></script>        <!-- download in parallel, run after parsing, in order -->
<script src="analytics.js" async></script>  <!-- download in parallel, run ASAP, any order -->
<script type="module" src="main.js"></script>  <!-- deferred by default -->
```

Related features: `rel="preload"` (Baseline widely available) fetches a known-needed resource early; `rel="modulepreload"` for ES modules; `fetchpriority` (Baseline since October 2024) nudges priority; `blocking="render"` (deliberately render-blocking) is **not Baseline** (no Firefox); `<script type="speculationrules">` prerendering of the next page is **Chromium-only**. HTTP **103 Early Hints** lets a server send `Link: rel=preload` headers before the HTML is ready.

Common mistakes

- Preloading everything — preloads compete with truly critical requests. Preload only late-discovered critical resources (a font referenced from CSS, a hero image set in CSS or JS).
- No `crossorigin` on font preloads → downloaded twice.
- Scripts at the end of `<body>` *and* `defer` — fine, but `defer` in `<head>` starts the download earlier.
- Inlining **all** CSS — uncacheable across pages, bloats every HTML response.

In the interview

30-second answer: "CSS in the head is render-blocking and classic scripts are parser-blocking; a script also waits for pending CSS. So I keep critical CSS small or inline it, load the rest without blocking, `defer` scripts (or `async` for independent ones), make the LCP image discoverable in the HTML with `fetchpriority=high`, preconnect to key origins, and avoid chains like CSS `@import`."

- *`async` vs `defer`?* → Both download in parallel; `defer` runs after parsing, in order; `async` runs on arrival, any order, possibly mid-parse.
- *Where should `<script>` go?* → In `<head>` with `defer` (or `type="module"`), so the download starts early without blocking the parser.

14.3 Core Web Vitals: LCP, INP, CLS

What it is

Core Web Vitals are Google's three user-experience metrics for loading, interactivity and visual stability, measured on **real visits** ("field data", vs "lab data" from test runs). A page passes when the **75th percentile** of visits (mobile and desktop separately) is "good".

Metric	Question it answers	Good	Needs improvement	Poor
LCP — Largest Contentful Paint	When did the main content appear?	≤ 2.5 s	2.5–4.0 s	> 4.0 s
INP — Interaction to Next Paint	How quickly does the page respond to clicks, taps and key presses?	≤ 200 ms	200–500 ms	> 500 ms
CLS — Cumulative Layout Shift	How much does content jump unexpectedly?	≤ 0.1	0.1–0.25	> 0.25

INP **replaced First Input Delay (FID)** in **March 2024**. FID measured only the *delay before the first* interaction's handler started; INP measures the *whole* time to the next painted frame, for (almost) *all* interactions.

How each is computed

LCP

1. Candidates: `<img>`, `<image>` inside SVG, `<video>` (poster or first frame), elements with CSS `background-image: url()`, and text-containing block elements.
2. Each larger painted candidate emits a new `largest-contentful-paint` entry; the **last** before the user's first input (click, tap, key press, scroll) is the LCP.
3. It splits into **TTFB** → **resource load delay** (until the image request starts) → **resource load duration** → **element render delay**.

INP

1. For each discrete **interaction** (click/tap = pointerdown + pointerup + click; key press = keydown + keyup; hover and scroll don't count), **latency** = **input delay** (the main thread is busy before your handlers can run) + **processing time** (your handlers) + **presentation delay** (style, layout, paint to the next frame).
2. The page's INP is the **worst** interaction latency — or, with many interactions, a high percentile (one outlier ignored per 50).
3. The Event Timing API reports it rounded to 8 ms; entries shorter than `durationThreshold` (default 104 ms, minimum 16 ms) aren't reported.

CLS

1. Each time a *visible* element moves between frames without user cause, a **layout shift** is scored **impact fraction** (share of viewport affected) × **distance fraction** (distance moved, relative to the viewport).
2. Shifts within 500 ms of input are excluded (`hadRecentInput`) — expected movement.
3. Shifts group into **session windows** (gaps < 1 s, window ≤ 5 s); CLS = the **largest window's sum**.

The experiment

Two versions of a product-guide page on a simulated phone (412×800) and slow network (CSS 300 ms, JS 800 ms, hero image 1800 ms):

	02_bad (html)	02_fixed (html)
Script	Sync in `<head>`	`defer`
Hero image	Injected by JS after `DOMContentLoaded`, `loading="lazy"`, no `width` / `height`	In the HTML, `width="800" height="533"`, `fetchpriority="high"`
Promo banner (appears after 1 s)	Inserted above the content into an empty `<div>`	Inserted into a slot with `min-height` reserved
Click handler	300 ms of synchronous work, then updates the UI	Updates the UI first, **yields** (pauses so the browser can paint and handle input), then does the 300 ms in 50 ms chunks

The INP fix:

```
const yieldToMain = () => globalThis.scheduler?.yield ? scheduler.yield() : new Promise(r => setTimeout(r, 0));

button.addEventListener('click', async () => {
  status.textContent = 'Saved ✓';           // 1. visible feedback first
  await yieldToMain();                        // 2. let the browser paint
  for (let i = 0; i < 6; i++) {               // 3. the same work, in small chunks
    busyFor(50);
    await yieldToMain();
  }
});
```

code/02_vitals.mjs installs three `PerformanceObserver`s (types `largest-contentful-paint`, `layout-shift`, and `event` with `durationThreshold: 16`, all `buffered`) before any page script, loads each page, clicks the button like a user, and computes the metrics as defined above (CLS = largest session window; INP = worst interaction, max duration per `interactionId`):

```
02_bad.html
  LCP 2648 ms  (element <img>, candidates: <p> @844, <img> @2648)  -> needs improvement
  CLS 0.195  (shifts: 0.069 @1833ms, 0.126 @2638ms)  -> needs improvement
  INP 304 ms  (slow events in the click: pointerdown, pointerup, click)  -> needs improvement
02_fixed.html
  LCP 1844 ms  (element <img>, candidates: <p> @340, <img> @1844)  -> good
  CLS 0.000  (shifts: none)  -> good
  INP < 16 ms  (slow events in the click: none)  -> good
```

(Timings vary by a few ms per run; the fixed INP is sometimes 16–24 ms.)

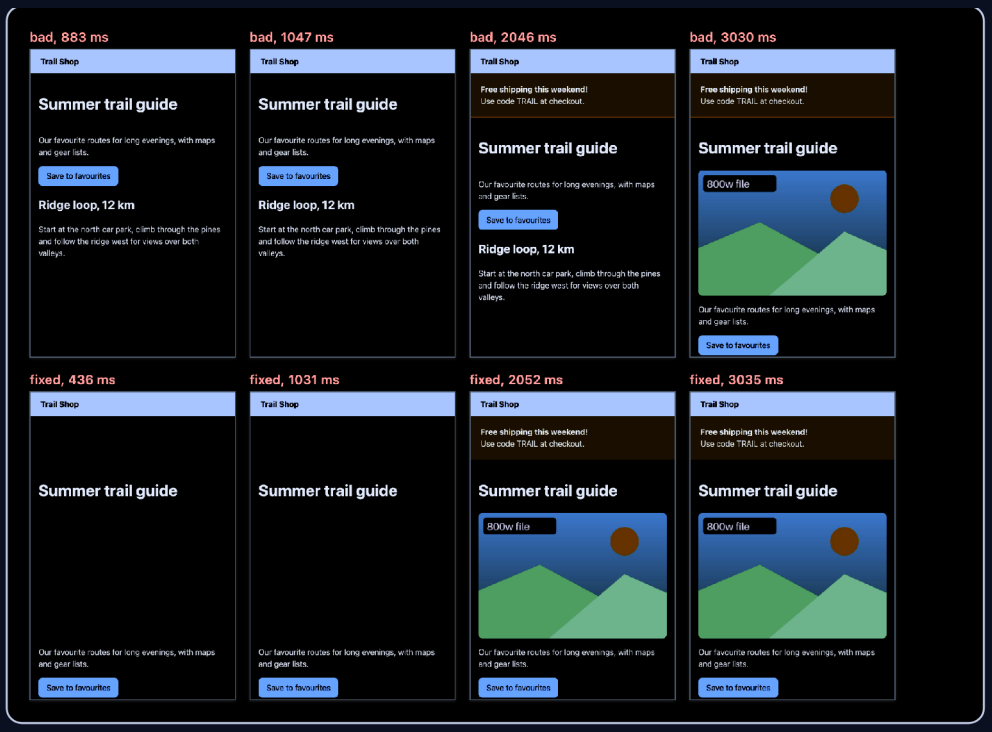


Figure 14.1 — code/make_figures.mjs : screenshots requested at ~0.4, 1, 2 and 3 s; labels show when Chrome delivered each frame (the bad page can't be captured before its first paint at ~880 ms).

- **LCP 2648 → 1844 ms.** The bad page's image request couldn't start until the blocking script (800 ms) ran and injected it — ~850 ms **resource load delay** before an 1800 ms download. In the fixed page the **preload scanner** (look-ahead parser) finds the `<img>` at once, so LCP ≈ download time. The remaining 1.8 s is **load duration** — next fix: a smaller image (`srcset`, AVIF/WebP, §4.3).
- **CLS 0.195 → 0.** Two shifts 0.8 s apart fell in one session window (0.069 + 0.126). `width` / `height` give the image an `aspect-ratio` (its box is reserved, so loading moves nothing) and `min-height` holds the banner slot.
- **INP 304 ms → under 16 ms.** All three click events are one interaction, and the 300 ms loop delayed the next paint for all of them. After the fix feedback paints immediately; the same work runs in chunks between frames.

Two measurement gotchas. (1) An early fixed page still shifted 15.7 px because its `height` said 500 for an 800×**533** image — wrong dimensions are almost as bad as none. (2) Headless Chrome reported no Event Timing entries for a button outside its real (smaller) window, so code/serve.mjs uses `--window-size=1200,1400`. Lab tooling has quirks — cross-check with field data.

Field data vs lab data

	Field (real users, "RUM")	Lab (synthetic)
Sources	Chrome UX Report (CrUX, 28-day rolling, public), Search Console's Core Web Vitals report, your own RUM with the `web-vitals` library	Lighthouse (in Chrome DevTools), PageSpeed Insights' lab section, WebPageTest, DevTools Performance panel, scripts like this chapter's
Strength	What users actually experience; what Google uses	Repeatable, debuggable, before release
INP	Measured	Needs a scripted interaction; Lighthouse's navigation mode reports **Total Blocking Time (TBT)** as a proxy

API support (web-features, September 2026): the LCP and Event Timing APIs are **Baseline newly available since December 2025** (Safari 26.2 added them). The **Layout Instability API** behind CLS is **Chromium-only**, so CLS field data comes from Chrome users. The metrics apply to all users; only measurement coverage differs.

In the interview

> **30-second answer:** "Core Web Vitals are LCP for loading — good is ≤ 2.5 s; INP for responsiveness — ≤ 200 ms; CLS for visual stability — ≤ 0.1, all at the 75th percentile of real visits. INP replaced FID in March 2024. I measure field data with CrUX or web-vitals and debug in the lab with Lighthouse and DevTools. Typical fixes: make the LCP image discoverable and high-priority, break up long tasks and yield to the main thread, and reserve space for images, ads and late content."

- *Why can't Lighthouse show INP on page load?* → INP needs real interactions; lab tools use TBT as a proxy or a scripted user flow.
- *Is a layout shift after a click counted?* → Not within 500 ms of the input (`hadRecentInput`).
- *Does a font swap count as CLS?* → Yes, if text reflows and moves content; match fallback metrics (`size-adjust`) or use `font-display: optional`.

14.4 Fixing each metric (and the other basics)

LCP checklist

1. **Find the LCP element** (DevTools Performance panel or `largest-contentful-paint` entries) and its four sub-parts (TTFB, load delay, load duration, render delay).
2. **TTFB**: caching (CDN, HTTP cache headers), no redirects (each costs a round trip), server-side rendering or static generation (HTML built at deploy time) for content pages.
3. **Load delay**: image in the HTML (not CSS/JS); `fetchpriority="high"`; never lazy; preload if it must be referenced from CSS.
4. **Load duration**: right-sized responsive images (`srcset` / `sizes`), AVIF/WebP, a CDN.
5. **Render delay**: remove render-blocking CSS/JS (14.2); don't hide content until JS runs (A/B-testing snippets, client-only rendering).

INP checklist

1. **Input delay**: break up **long tasks** (> 50 ms of uninterrupted main-thread work) during load; defer third-party scripts (ads, chat, analytics); avoid heavy timers colliding with input.
2. **Processing**: only the visible update in the handler; `await scheduler.yield()` (or `setTimeout`) before the rest; debounce input handlers (react when typing pauses); heavy work in a Web Worker (background JS thread).
3. **Presentation**: small DOM; no layout thrashing; `content-visibility: auto` for long off-screen sections (Baseline since September 2025); animate `transform` / `opacity` (§12.7).

CLS checklist

1. Always **`width`** **and** **`height`** (correct values) on images and videos, or CSS `aspect-ratio`.
2. **Reserve space** for ads, embeds, banners and late components (`min-height`, skeleton placeholders), or overlay them (`position: fixed`) so they don't push content.
3. Never insert content **above** existing content unless the user asked.
4. Fonts: `font-display: optional`, or tune the fallback with `size-adjust` / `ascent-override`.
5. Animate with `transform`, not `top` / `height` (transforms don't cause layout shifts).
6. Stay eligible for the **back/forward cache** (bfcache) — instant, shift-free back navigations (avoid `unload` handlers and `Cache-Control: no-store` where possible).

Other performance basics

Area	Practice
Transfer	Brotli/gzip/zstd, HTTP/2 or HTTP/3, long cache lifetimes for fingerprinted (content-hashed) files (`app.3f9a1c.css` , `Cache-Control: max-age=31536000, immutable`) — §1.5
Images	Formats and responsive images (§4.2); `loading="lazy"` below the fold
CSS	Remove unused CSS; avoid huge selectors on huge DOMs; `contain` / `content-visibility` for independent regions
JavaScript	Ship less; code-split per route (load pieces on demand); `defer` ; audit third parties
Budgets	Performance budgets (e.g. LCP < 2.5 s on a mid-range phone, JS < 200 KB compressed) checked in CI with Lighthouse CI

14.5 SEO basics

What it is

Search engine optimization (SEO) makes pages easy for search engines to **crawl** (fetch), **render** (Google runs JavaScript in an up-to-date Chromium), **index** (store and understand) and **rank**, and attractive to click. Content quality and relevance matter most — HTML can't make a thin page rank — but broken HTML *can* keep a good page out of the index. This is the HTML side, following Google Search Central.

The essentials, in the order a crawler meets them

Item	HTML	Notes
Crawl permission	`robots.txt` (rules for crawler bots) at the site root	A `Disallow` rule blocks **crawling**, not indexing: a blocked URL can still be indexed from links. To keep a page out, allow crawling and use `noindex`
Index permission	`<meta name="robots" content="noindex">` (or `X-Robots-Tag` header)	The classic accident: a staging `noindex` shipped to production
Title	`<title>Specific page title | Brand</title>`	The result headline (Google may rewrite it). Unique per page; no hard limit, but truncated by width — ~50–60 characters display fully
Description	`<meta name="description" content="…">`	Often the snippet; not a ranking signal; unique, specific; truncated around 150–160 characters
Canonical	`<link rel="canonical" href="https://…">`	Preferred URL among duplicates (tracking parameters like `?utm_source=`, `http` / `https`, `www`). A strong **hint**, not a directive (Google usually follows it). Absolute URLs recommended
Language	`<html lang="en">`; `<link rel="alternate" hreflang="de" href="…">`	`hreflang` links translations (include a self-reference and optionally `x-default`)
Mobile	`<meta name="viewport" content="width=device-width, initial-scale=1">`	Google indexes the **mobile** version (mobile-first indexing)
Keywords meta	`<meta name="keywords">`	**Ignored** by Google Search
Social previews	`og:title`, `og:image`, … (Open Graph)	Link previews in social apps; not a Google ranking signal
Sitemap	`sitemap.xml` (XML list of the site's URLs), listed in robots.txt / Search Console	Helps discovery of new or deep pages

Semantic HTML is SEO

Search engines read structure like assistive technology does (§3.1):

- **Headings** describe the outline; one clear `<h1>` is a helpful convention (Google says multiple `<h1>`s aren't a problem); don't skip levels.
- **Links** must be real `<a href>` (Google follows `href`, not `onclick`) with **descriptive text** — "Ridge loop route and map", not "click here".
- **Images**: `alt` text for image search and accessibility; descriptive file names.
- **Landmarks and** `<article>` / `<time>` identify the main content and dates.
- Content must be in the HTML or reliably rendered; content needing a click to load may never be seen.

Structured data (JSON-LD)

Structured data describes the page with the **schema.org** vocabulary (types like Article, Product, Person and their properties) in machine-readable form, making it eligible for **rich results** (breadcrumbs, product price and rating, recipe cards, article dates). Google supports JSON-LD (JSON for Linked Data), Microdata and RDFa (attribute-based), and **recommends JSON-LD**: a `<script type="application/ld+json">` block separate from the visible markup.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Summer Trail Guide: 5 Evening Hikes Near Denver",
  "image": ["https://www.trailshop.example/img/hills-1200.jpg"],
  "datePublished": "2026-06-01T08:00:00-06:00",
  "dateModified": "2026-06-15T09:30:00-06:00",
  "author": [{ "@type": "Person", "name": "Maya Ortiz", "url": "https://www.trailshop.example/team/maya" }]
}
</script>
```

Rules: describe what's **visible** on the page; include each rich-result type's **required** properties (Article has none; the above are Google's recommended set); valid JSON (double quotes, no trailing commas). Structured data makes a page *eligible* — it **doesn't guarantee** rich results and isn't a direct ranking boost. Google restricts or retires types over time (FAQ and HowTo results were cut back in 2023), so check the current Search Gallery. Test with the **Rich Results Test** and **Schema Markup Validator**.

The experiment

`code/03_seo_audit.mjs` is a small jsdom on-page checker (parsing HTML like a crawler's first pass). It audits `code/03_seo_bad.html` — `div` soup with an accidental `noindex`, a relative canonical, two `<h1>`s, no `alt`, "Click here" and JSON-LD with a trailing comma — and `code/03_seo_good.html`:

```
03_seo_bad.html
  FAIL  doctype present (standards mode)
  FAIL  <html lang> = null
  FAIL  <title> 4 chars: "Home"
  FAIL  meta description missing
  FAIL  meta viewport (mobile-friendly)
  FAIL  robots: "noindex"
  FAIL  canonical: /guide (absolute URL recommended)
  NOTE  meta keywords present — ignored by Google, harmless but useless
  FAIL  one <h1> (found 2) — a clarity convention, not a Google rule
  FAIL  heading levels 1→1→4 (skips a level)
  FAIL  <main> landmark
  FAIL  images without alt: 1
  FAIL  vague link text: "Click here"
  FAIL  JSON-LD does not parse: Expected double-quoted property name in JSON at position 95 (line 2 column 95)
  score: 0/13

03_seo_good.html
  PASS  doctype present (standards mode)
  PASS  <html lang> = "en"
  PASS  <title> 60 chars: "Summer Trail Guide: 5 Evening Hikes Near Denver | Trail Shop"
  PASS  meta description 136 chars
  PASS  meta viewport (mobile-friendly)
  PASS  robots: "(none = index, follow)"
  PASS  canonical: https://www.trailshop.example/guides/summer-trails (absolute URL recommended)
  ...   (headings, <main>, alt text and link text all pass)
  PASS  JSON-LD Article: recommended properties present
  PASS  JSON-LD BreadcrumbList: recommended properties present
  score: 14/14
```

The most damaging line in the bad page is `noindex` — every other fix is wasted while it's there. The trailing comma is the second classic: the whole JSON-LD block is silently ignored.

Performance and SEO

Core Web Vitals are part of Google's page-experience signals: good vitals help, but relevance dominates — a fast page with poor content won't outrank a slower, better answer. Treat performance as a user-experience goal, SEO a side benefit.

Common mistakes

- Client-only rendering of content pages with empty initial HTML (rendering may be delayed; other crawlers and social previews may not run JS). Prefer server rendering or static generation.
- Blocking CSS/JS in `robots.txt` → Google can't render the page properly.
- The same `<title>`/description everywhere; every page canonical to the home page.
- Links built with `<span onclick>` or `<a>` without `href`.

In the interview

> **30-second answer:** "For SEO the HTML needs a unique, descriptive title and meta description, a canonical URL, `lang` and viewport, no accidental `noindex`, semantic structure — headings, landmarks, real links with descriptive text, alt text — and JSON-LD structured data using schema.org to become eligible for rich results. Content should be server-rendered or reliably renderable, and good Core Web Vitals help as part of page experience."

14.6 Interview questions

Forty-seven common front-end interview questions with short answers and links to the full explanation. Cover the answer, say yours aloud, then check.

HTML and accessibility

1. **What does `<!doctype html>` do?** → Standards mode (without it: quirks mode with legacy layout rules). §2.2
2. **Why is `<meta charset="utf-8">` first in `<head>`?** → It must be within the first 1024 bytes so the parser decodes the rest correctly. §2.3
3. **Block vs inline elements?** → Block boxes start on a new line and take the available width; inline boxes flow within text. It's CSS `display`, not the element "type". §7.4
4. **`<section>` vs `<article>` vs `<div>`?** → `<article>` is self-contained (could be syndicated on its own); `<section>` is a thematic group with a heading; `<div>` has no meaning. §3.2
5. **What are landmarks and why do they matter?** → `header`, `nav`, `main`, `aside`, `footer` (and named `form` / `section`) map to ARIA landmark roles screen-reader users jump between. §3.2
6. **What is the first rule of ARIA?** → Don't use ARIA if a native element does the job; no ARIA is better than bad ARIA. §3.6
7. **How do you write good alt text?** → Describe the image's purpose in context; `alt=""` for decorative images; never omit `alt`. §3.5
8. **`<button>` vs `<a>`?** → Links navigate (have `href`); buttons perform actions. Both are natively keyboard-accessible; a `div` with `onclick` is not. §3.7
9. **How do you label a form field accessibly?** → `<label for="id">` (or wrap the input); group radios with `fieldset` / `legend`; link errors with `aria-describedby`. §3.8, §5.7
10. **GET vs POST forms?** → GET puts fields in the URL (bookmarkable, for searches); POST sends them in the request body (changes, sensitive data). §5.2
11. **How do `srcset` and `sizes` work?** → `srcset` lists candidate files with widths; `sizes` gives the displayed width; the browser picks the smallest adequate file for the screen's density. §4.3
12. **When do you use `<picture>`?** → Art direction (different crops per breakpoint) or format fallbacks (AVIF → WebP → JPEG). §4.3
13. **What does WCAG AA require for text contrast?** → 4.5:1 for normal text, 3:1 for large text and UI components. §3.9

CSS fundamentals

14. **How is specificity calculated?** → (IDs, classes/attributes/pseudo-classes, types/pseudo-elements), compared left to right; inline styles beat selectors; `!important` changes the origin ordering. §6.4

15. **What decides which declaration wins?** → Origin and importance, context, inline style, cascade layer, specificity, (for `@scope` rules) scope proximity, then order of appearance. §6.5
16. **Pseudo-class vs pseudo-element?** → `:hover` selects an element in a state; `::before` creates/selects a part of an element. §6.3
17. **Which properties inherit?** → Mostly text properties (colour, font, line-height); box properties (margin, border, width) don't. `inherit`, `initial`, `unset`, `revert` control it. §6.6
18. **`em` vs `rem`?** → `em` is relative to the element's own font size (the parent's for `font-size`); `rem` to the root's. Use `rem` for predictable scaling. §6.7
19. **Custom properties vs Sass variables?** → Live, inherited values that can change per element and at runtime; Sass variables disappear at compile time. §6.9, §13.12
20. **`:is()` vs `:where()`?** → Same matching; `:is()` takes its most specific argument's specificity, `:where()` is zero. §13.2
21. **What does `:has()` enable?** → Selecting an element by what it contains or what follows it — a parent/previous-sibling selector. §13.3
22. **How do cascade layers help?** → They order groups of rules before specificity is considered, ending specificity wars with third-party CSS. §13.4

Box model and layout

23. **Explain the box model and `box-sizing`.** → Content + padding + border + margin; `content-box` sizes the content only, `border-box` includes padding and border. §7.1, §7.2
24. **What is margin collapsing?** → Adjacent vertical margins of blocks in normal flow combine into the larger; not in flex/grid or across a BFC boundary (Q27). §7.3
25. **Explain `position` values.** → `static` (normal), `relative` (offset, keeps its space), `absolute` (out of flow, nearest positioned ancestor), `fixed` (viewport), `sticky` (relative until a threshold, then sticks within its container). §7.6
26. **Why doesn't my `z-index` work?** → It applies only to positioned (or flex/grid) items and is compared within one stacking context; a parent with `opacity < 1`, `transform` etc. creates a new one. §7.8
27. **What is a block formatting context (BFC)?** → An independent layout region: contains floats, prevents margin collapse with children; created by `display: flow-root`, flex/grid items, `overflow` other than `visible` / `clip`, etc. §7.5
28. **What does `flex: 1` mean?** → `flex: 1 1 0%`: grow equally from a zero basis, allowed to shrink. §8.4
29. **Why does a flex item with long text overflow instead of shrinking?** → Its automatic minimum is `min-width: auto` (its content); set `min-width: 0` (a scroll container via `overflow: hidden` / `auto` also works; `overflow: clip` doesn't). §8.4
30. **How do you center something horizontally and vertically?** → `display: grid; place-items: center` (or flex with `justify-content` and `align-items: center`). §8.7
31. **Flexbox or Grid?** → Flexbox for one-dimensional content-driven layout; Grid for two-dimensional, layout-driven structure. §9.8
32. **`auto-fill` vs `auto-fit`?** → Both create as many tracks as fit; `auto-fit` collapses empty ones so items stretch. §9.3
33. **What is subgrid for?** → Letting nested items align to the parent grid's tracks (card titles aligned across cards). §9.7

Responsive, visual and motion

34. **What does the viewport meta tag do?** → Tells mobile browsers to lay out at device width instead of a ~980px virtual desktop. §10.2
35. **Mobile-first — what and why?** → Base styles for small screens, `min-width` queries add complexity; simpler CSS, faster phones. §10.4
36. **Media queries vs container queries?** → Media queries respond to the viewport; container queries to a component's container — reusable components. §10.6
37. **How does `clamp()` give fluid typography?** → `clamp(min, preferred, max)` with a viewport-relative preferred value (plus `rem` so zoom still works). §10.7
38. **How do you load a web font without invisible text?** → `font-display: swap` (or `optional`), preload the WOFF2, subset it. §11.2

39. **Which properties are cheap to animate and why?** → `transform` and `opacity`: the compositor animates them without layout or paint. §12.7
40. **How do you respect users who get motion sickness?** → Wrap non-essential motion in `@media (prefers-reduced-motion: no-preference)` or reduce it under `reduce`. §12.9

Architecture, performance and SEO

41. **BEM vs CSS Modules vs Tailwind?** → Convention (flat unique class names) vs build-time renamed classes vs generated single-purpose utilities; all fight the global namespace. §13.11
42. **Dark mode with a user toggle?** → Semantic tokens with `light-dark()`, `color-scheme: light dark`, a `data-theme` override pinning `color-scheme`, saved in localStorage and applied before first paint. §13.14
43. **What happens when you type a URL and press Enter?** → DNS, TCP/TLS, HTTP request/response, parsing, DOM/CSSOM, render tree, layout, paint, composite. §1.6
44. **What are render-blocking resources?** → Stylesheets (block paint) and synchronous scripts (block parsing); fix with critical CSS, `defer` / `async`. §14.2
45. **Core Web Vitals and their thresholds?** → LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75. §14.3
46. **How do you prevent layout shift from images?** → `width` / `height` attributes (correct ratio) or `aspect-ratio`. §14.4
47. **What HTML matters for SEO?** → Title, description, canonical, lang, viewport, no stray `noindex`, headings, real links, alt, JSON-LD. §14.5

14.7 Cheat sheets

Selectors

Selector	Matches	Specificity
`*`	every element	(0,0,0)
`p`	type	(0,0,1)
`.card`	class	(0,1,0)
`#main`	id	(1,0,0)
`[type="email"]`, `[href^="https"]`, `[href$=".pdf"]`, `[class*="icon"]`, `[lang|="en"]`	attribute: equals / starts / ends / contains / language prefix	(0,1,0)
`A B`	descendant	sum
`A > B`	child	sum
`A + B`	next sibling	sum
`A ~ B`	any later sibling	sum
`:hover :focus :focus-visible :focus-within :active :visited`	user action state	(0,1,0) each
`:checked :disabled :required :invalid :user-invalid :placeholder-shown`	form state	(0,1,0)
`:first-child :last-child :nth-child(2n+1) :nth-of-type(3) :only-child :empty`	structure	(0,1,0)
`:nth-child(2 of .item)`	nth among matches	(0,1,0) + argument
`:not(X) :is(X) :has(X)`	logic	most specific argument
`:where(X)`	logic	(0,0,0)
`::before ::after ::marker ::placeholder ::selection ::first-line ::backdrop`	parts	(0,0,1)
`&` (nesting)	parent rule's selector	as `:is(parent)`

Details: §6.2, §6.3, §13.1–13.3.

Specificity and the cascade

- Score = (A = IDs, B = classes + attributes + pseudo-classes, C = types + pseudo-elements); compare A, then B, then C. `*` and combinators add nothing.
- The cascade, strongest criterion first (UA = user agent, i.e. the browser's own default styles): **origin & `!important`** (transition > `!important` UA > `!important` user > `!important` author > animation > author > user > UA) → **context** (shadow DOM) → **inline `style`** → **layers** (later wins; unlayered last; reversed for `!important`) → **specificity** → **scope proximity** (`@scope`) → **order of appearance**.
- Keywords: `inherit` (parent's value), `initial` (spec default — the value the CSS specification defines), `unset` (inherit if inherited property, else initial), `revert` (browser/user value), `revert-layer` (previous layers' value).

Worked examples: `#nav .item a` = (1,1,1); `ul li:nth-child(2)` = (0,1,2); `a:not(.btn, #x)` = (1,0,1); `:where(#x) .y` = (0,1,0). §6.4

Box model

```
margin  →  border  →  padding  →  content
content-box:  width = content          total = width + padding + border (+ margin)
border-box:   width = content + padding + border
```

- Vertical margins between blocks in normal flow **collapse** (largest wins); not in flex/grid, not through padding/border, not inside a BFC. §7.3
- Percentage padding/margin are relative to the **containing block's inline size** — its width in horizontal text, even for top/bottom.
- `display`: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `flow-root` (new BFC), `contents` (box removed, children stay — careful with accessibility), `none` (removed from layout and accessibility tree).
- Positioning: `relative` → offset; `absolute` → nearest positioned ancestor; `fixed` → viewport (or a transformed ancestor); `sticky` → scroll threshold. `inset: 0` = `top/right/bottom/left: 0`.

Flexbox

Container	Values (default first)
`display`	`flex` | `inline-flex`
`flex-direction`	`row` | `row-reverse` | `column` | `column-reverse`
`flex-wrap`	`nowrap` | `wrap` | `wrap-reverse`
`flex-flow`	shorthand: direction + wrap
`justify-content` (main axis)	`flex-start` / `normal` | `center` | `flex-end` | `space-between` | `space-around` | `space-evenly`
`align-items` (cross axis, per line)	`stretch` / `normal` | `flex-start` | `center` | `flex-end` | `baseline`
`align-content` (lines, when wrapping)	`normal` | `flex-start` | `center` | `space-between` | `stretch` …
`gap`, `row-gap`, `column-gap`	spacing between items

Item	Values
`flex-grow`	`0` — share of free space
`flex-shrink`	`1` — share of overflow to absorb (weighted by basis)
`flex-basis`	`auto` — starting size before grow/shrink
`flex`	`initial` = `0 1 auto`; `auto` = `1 1 auto`; `none` = `0 0 auto`; `1` = `1 1 0%`
`align-self`	overrides `align-items` for one item
`order`	`0` — visual order only (keyboard/reading order unchanged)
`min-width: 0`	lets items shrink below their content (`overflow: clip` doesn't; `hidden` / `auto` do)

Chapter 8

Grid

Container	Example
`grid-template-columns` / `-rows`	`200px 1fr 2fr`, `repeat(3, 1fr)`, `repeat(auto-fit, minmax(16rem, 1fr))`
`grid-template-areas`	`"head head" "side main" "foot foot"`
`grid-auto-rows` / `-columns`	size of implicit tracks: `minmax(100px, auto)`
`grid-auto-flow`	`row` | `column` | `dense`
`gap`	`1rem 2rem` (row column)
`justify-items` / `align-items` / `place-items`	align items inside their cells
`justify-content` / `align-content`	align the whole grid inside the container

Item	Example
`grid-column` / `grid-row`	`1 / 3`, `2 / span 2`, `1 / -1` (full width)
`grid-area`	`main` (named area) or `1 / 1 / 3 / 3`
`justify-self` / `align-self` / `place-self`	per-item alignment
`grid-template-columns: subgrid`	reuse the parent's tracks

Track sizes: `px` / `rem`, `%`, `fr` (share of free space), `auto`, `min-content`, `max-content`, `fit-content(300px)`, `minmax(min, max)`. Chapter 9

Units

Unit	Relative to	Typical use
`px`	CSS pixel (1/96 in; scales with zoom and device pixel ratio)	borders, small fixed details
`rem`	root font size (16px default)	font sizes, spacing
`em`	own font size (parent's for `font-size`)	padding that scales with text
`%`	containing block (varies by property)	fluid widths
`ch` / `ex` / `lh`	width of "0" / x-height / line height	`max-width: 65ch` for readable lines
`vw` / `vh`	1% of viewport width / height	full-screen sections
`svh` / `lvh` / `dvh`	small / large / dynamic viewport height (mobile toolbars)	`min-height: 100dvh`
`vi` / `vb`	viewport inline / block size	writing-mode-aware
`cqi` / `cqw` / `cqb`	container query inline / width / block	component-relative type
`fr`	free space in a grid	grid tracks
`deg` `turn` `ms` `s`	angles, time	transforms, transitions

§6.7, §10.2

Responsive patterns

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
/* mobile-first breakpoints (range syntax) */
.layout { display: grid; gap: 1rem; }
@media (width >= 48rem) { .layout { grid-template-columns: 16rem 1fr; } }

/* intrinsic grid, no breakpoints */
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr)); gap: 1rem; }

/* component responds to its container */
.card-wrap { container-type: inline-size; }
@container (inline-size >= 30rem) { .card { display: flex; } }

/* fluid type and space */
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }

/* user preferences */
@media (prefers-color-scheme: dark) { /* or use light-dark() tokens */ }
@media (prefers-reduced-motion: reduce) { * { animation: none; } }

/* images */
img { max-width: 100%; height: auto; }
```

```
<img src="a-800.jpg" srcset="a-400.jpg 400w, a-800.jpg 800w, a-1600.jpg 1600w"
     sizes="(width >= 48rem) 50vw, 100vw" width="800" height="533" alt="…">
```

Chapter 10

HTML head starter

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title | Site</title>
  <meta name="description" content="What this page offers, in one or two sentences.">
  <link rel="canonical" href="https://example.com/page">
  <meta name="color-scheme" content="light dark">
  <link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
  <link rel="stylesheet" href="/css/site.3f9a1c.css">
  <script src="/js/app.8b2e0d.js" defer></script>
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
</head>
```

14.8 Glossary

Alphabetical; each term links to the section that explains it.

- `!important` — Flag that moves a declaration to a higher-priority "important" origin level. §6.5
- `@container` — Container query rule. §10.6
- `@font-face` — Declares a web font and its files. §11.1
- `@import` — Loads another stylesheet (can add a layer). §6.1
- `@keyframes` — Named animation timeline. §12.5
- `@layer` — Cascade layer. §13.4
- `@media` — Media query rule. §10.3
- `@property` — Registers a typed custom property. §13.14
- `@scope` — Limits rules to a subtree, optionally with a lower bound. §13.5
- `@starting-style` — Style to transition *from* when an element first renders. §12.8

- `@supports` — Feature query. §6.1
- **Absolute positioning** — Out of flow, placed relative to the nearest positioned ancestor. §7.6
- **Absolute URL** — URL with scheme and host. §1.2
- **Accessibility tree** — The browser's tree of roles, names and states for assistive tech. §3.4
- **Accessible name** — What a screen reader announces for an element. §3.4
- `alt` text — Text alternative for an image. §3.5
- **ARIA** — Attributes that add roles/states/properties for assistive technology. §3.6
- `aria-describedby` — Links an element to its description (e.g. an error). §5.7
- `aspect-ratio` — Preferred width/height ratio of a box; `<img width height>` sets it automatically. §4.1
- `async` (script) — Download in parallel, run as soon as available. §14.2
- **Attribute** — Name/value pair on an element. §2.1
- **Attribute selector** — `[name="value"]` and variants. §6.2
- `autocomplete` — Hints the browser what data a field wants. §5.6
- **AVIF** — Modern image format with strong compression. §4.2
- **Baseline** — web-features status: works in all major browsers (newly / widely available). §1.1
- **BEM** — Block__Element--Modifier naming convention. §13.7
- **bfcache** — Back/forward cache: instant back navigations. §14.4
- **Block axis** — Direction in which blocks/lines stack. §13.6
- **Block formatting context (BFC)** — Independent layout region for block boxes. §7.5
- **Block-level element** — Starts on a new line and fills the width. §7.4
- `border-box` — `box-sizing` in which width includes padding and border. §7.2
- **Box model** — Content, padding, border, margin. §7.1
- **Breakpoint** — Width at which a layout changes. §10.3
- **Canonical URL** — Preferred URL among duplicates (`rel="canonical"`). §14.5
- **Canvas** — Scriptable bitmap drawing surface. §4.9
- **Cascade** — Algorithm that picks the winning declaration. §6.5
- **Cascade layer** — Named, ordered group of rules. §13.4
- **Character reference** — `&`, `©` — escapes in HTML. §2.9
- `clamp()` — Value clamped between a min and max. §10.7
- **CLS** — Cumulative Layout Shift — visual stability metric. §14.3
- `color-mix()` — Mixes two colours in a colour space. §11.4
- `color-scheme` — Declares supported light/dark schemes. §10.5
- **Combinator** — Relationship between selectors (space, `>`, `+`, `~`). §6.2
- **Composite** — Final step: combining layers into the screen image. §1.7
- **Compositor-only properties** — `transform`, `opacity` — animate without layout/paint. §12.7
- **Computed value** — Value after cascade and inheritance, before layout. §6.10
- **Constraint Validation API** — JS API behind built-in form validation. §5.5
- **Container query** — Styles based on a container's size. §10.6
- **Containing block** — Box that percentages and positions are resolved against. §7.6
- `content-visibility` — Skips rendering off-screen content. §14.4
- **Core Web Vitals** — LCP, INP and CLS. §14.3
- **Critical CSS** — CSS needed for the first screen, often inlined. §14.2
- **Critical rendering path** — Work required before the first render. §14.1
- **CrUX** — Chrome UX Report — public field data. §14.3
- **CSS Modules** — Build-time locally scoped class names. §13.9
- **CSS-in-JS** — Styles authored in JavaScript. §13.10
- **CSSOM** — CSS Object Model — parsed stylesheets. §1.7
- `cubic-bezier()` — Custom easing curve. §12.2
- **Custom property** — `--name` value that inherits and can change at runtime. §6.9
- `defer` (script) — Download in parallel, run in order after parsing. §14.2
- **Design token** — Named design value (colour, space) stored as a variable. §13.14

- **Device pixel ratio** — Physical pixels per CSS pixel. §4.3
- **DevTools** — Browser developer tools. §1.8
- `<dialog>` — Native modal/non-modal dialog element (focus handling built in). §3.7
- `display` — Outer/inner display type of a box. §7.4
- **DNS** — Resolves domain names to IP addresses. §1.3
- **Doctype** — `<!doctype html>` — standards mode. §2.2
- **DOM** — Document Object Model — tree of the parsed HTML. §1.7
- **Donut scope** — `@scope (root) to (limit)` excluding an inner region. §13.5
- `dvh` — Dynamic viewport height unit. §10.2
- **Easing** — Speed curve of an animation (timing function). §12.2
- **Element** — Start tag, content, end tag. §2.1
- `em` — Unit relative to font size. §6.7
- **Event Timing API** — Reports input event latencies (INP). §14.3
- **FCP** — First Contentful Paint. §14.1
- `fetchpriority` — Hint to raise/lower a resource's priority. §4.4
- **Field data** — Metrics from real users. §14.3
- `<fieldset>` / `<legend>` — Group of related controls and its caption. §5.4
- `<figure>` / `<figcaption>` — Self-contained content with caption. §4.5
- `animation-fill-mode` — Whether keyframe styles apply before the animation starts / after it ends. §12.5
- `flex` shorthand — grow, shrink, basis. §8.4
- `flex-basis` — Item's starting main size. §8.3
- **Flex container / item** — Element with `display: flex` / its children. §8.1
- **Float** — Legacy layout: box shifted left/right, text wraps. §7.9
- **FLIP** — First-Last-Invert-Play animation technique. §12.7
- **Focus indicator** — Visible outline showing keyboard focus. §3.7
- `:focus-visible` — Focus that should show an indicator (keyboard). §3.7
- `font-display` — How text renders while a web font loads. §11.2
- **FOUC** — Flash of unstyled content. §14.1
- `fr` — Fraction of free space in a grid. §9.2
- `gap` — Space between flex/grid items. §8.6
- **Gradient** — `linear-/radial-/conic-gradient()` images. §11.5
- **Grid area** — Named or line-bounded region. §9.4
- **Grid line / track / cell** — Dividing lines / rows-columns / single slot. §9.1
- `:has()` — Relational ("parent") pseudo-class. §13.3
- **Heading levels** — `h1` – `h6` outline. §3.3
- `hreflang` — Links language versions of a page. §14.5
- **HSL / OKLCH** — Colour notations (hue-based, perceptual). §6.8
- **HTTP** — Request/response protocol of the web. §1.5
- **HTTP/2, HTTP/3** — Multiplexed HTTP versions (HTTP/3 over QUIC). §1.4
- `<iframe>` — Embedded browsing context. §4.7
- **Implicit grid** — Tracks created automatically for extra items. §9.5
- **INP** — Interaction to Next Paint — responsiveness metric. §14.3
- **Inheritance** — Children taking a parent's computed value. §6.6
- **Inline axis** — Direction text runs within a line. §13.6
- **Inline element** — Flows within a line of text. §7.4
- **Input types** — `email`, `number`, `date`, …. §5.3
- **Intrinsic layout** — Responsive without breakpoints (`auto-fit`, `min()`). §10.9
- `:is()` — Matches any selector in a list (max specificity). §13.2
- **JSON-LD** — JSON format for structured data. §14.5
- **Keyboard trap** — Focus that can't leave a component (WCAG failure). §3.7

- **Lab data** — Metrics from synthetic tests. §14.3
- **Landmark** — Region role (`banner`, `navigation`, `main`, …). §3.2
- **Layout (reflow)** — Computing box sizes and positions. §1.7
- **Layout shift** — Unexpected movement of visible content. §14.3
- **Lazy loading** — `loading="lazy"`: fetch when near the viewport. §4.4
- **LCP** — Largest Contentful Paint — loading metric. §14.3
- `light-dark()` — Picks a light or dark value by colour scheme. §13.14
- **Lighthouse** — Lab auditing tool (performance, a11y, SEO). §14.3
- **Link (`<a>`)** — Hyperlink; `href`, `target`, `rel`. §2.8
- **Lists** — `ul`, `ol`, `dl`. §2.7
- **Logical properties** — Flow-relative properties (`margin-inline-start`). §13.6
- **Long task** — Main-thread task over 50 ms. §14.4
- **Margin collapsing** — Adjacent vertical margins combine. §7.3
- **Media query** — Condition on the device/viewport/preferences. §10.3
- **Meta description** — Page summary for search snippets. §14.5
- `min-content` / `max-content` — Intrinsic sizes. §9.2
- `minmax()` — Track size range. §9.2
- **Mobile-first** — Base styles for small screens, enhance upward. §10.4
- **Nesting (CSS)** — Rules inside rules; `&`. §13.1
- `noindex` — Robots directive keeping a page out of the index. §14.5
- **normalize.css** — Stylesheet making browser defaults consistent. §13.13
- **Normal flow** — Default layout of block and inline boxes. §7.5
- `object-fit` — How replaced content fills its box. §11.9
- **Open Graph** — `og:` meta tags for social previews. §14.5
- `order` — Visual order of flex/grid items. §8.2
- **Origin (cascade)** — User-agent, user or author stylesheet. §6.5
- **Outline** — Line outside the border that doesn't take space. §11.6
- `overflow` — What happens to content that doesn't fit. §7.7
- **Paint** — Filling in pixels for boxes. §1.7
- `PerformanceObserver` — API to receive performance entries. §14.3
- `<picture>` — Art direction and format fallbacks. §4.3
- `place-items` — Shorthand for `align-items` + `justify-items`. §9.6
- **Positioned element** — `position` other than `static`. §7.6
- **PostCSS** — Plugin-based CSS transformer. §13.12
- `preconnect` — Early connection to another origin. §14.2
- `prefers-color-scheme` — Media feature for light/dark preference. §10.5
- `prefers-reduced-motion` — Media feature for reduced motion. §12.9
- `preload` — Fetch a known-critical resource early. §14.2
- **Preload scanner** — Look-ahead parser that finds resources early. §14.3
- **Preprocessor** — Language compiled to CSS (Sass, Less). §13.12
- **Pseudo-class** — `:state` selector (`:hover`, `:checked`). §6.3
- **Pseudo-element** — Selects or creates a part of an element (`::before`, `::marker`). §6.3
- **Quirks mode** — Legacy rendering mode without a doctype. §2.2
- `rel` — Link relationship (`noopener`, `canonical`, `preload`). §2.8
- **Relative colour syntax** — Derive a colour from another's channels: `oklch(from var(--c) l c calc(h + 180))`. §11.4
- `rem` — Unit relative to root font size. §6.7
- **Render tree** — Styled boxes that will be displayed. §1.7
- **Render-blocking** — Delays the first paint (stylesheets). §14.2
- `repeat()` — Repeats track definitions. §9.3
- **Reset (CSS)** — Rules neutralising browser defaults. §13.13

- **Responsive design** — Layouts that adapt to any screen and setting. §10.1
- `revert-layer` — Roll back to earlier layers' value. §13.4
- **Rich results** — Enhanced search results from structured data. §14.5
- `robots.txt` — Crawl rules for bots. §14.5
- **Role** — ARIA/semantic purpose of an element. §3.6
- **Sass** — The most used CSS preprocessor. §13.12
- `sandbox` — Restricts an iframe's capabilities. §4.7
- **schema.org** — Vocabulary for structured data. §14.5
- **Screen reader** — Software that speaks or brailles the UI. §3.1
- `<search>` — Landmark element for search/filter forms (widely available since April 2026). §3.2
- **Scroll-driven animation** — Animation progress linked to scroll. §12.11
- **Selector** — Pattern choosing elements to style. §6.2
- **Semantic HTML** — Elements chosen for meaning. §3.1
- **SEO** — Search engine optimization. §14.5
- **Session window (CLS)** — Group of layout shifts < 1 s apart, ≤ 5 s. §14.3
- **Shorthand property** — Sets several longhands at once (`margin` , `flex` , `inset`). §7.1
- **Sitemap** — XML list of URLs for crawlers. §14.5
- `sizes` — Displayed image width per condition. §4.3
- **Specificity** — Weight of a selector (A,B,C). §6.4
- `srcset` — Candidate image files. §4.3
- **Stacking context** — Group painted together along z. §7.8
- **Sticky positioning** — Relative until a scroll threshold, then fixed within its container. §7.6
- **Structured data** — Machine-readable page description. §14.5
- **Style query** — `@container style(--x: y)` — query a container's custom property (Baseline since May 2026). §10.6
- **Subgrid** — Nested grid reusing parent tracks. §9.7
- **SVG** — Vector image format written in XML. §4.8
- **Table semantics** — `caption` , `thead` , `th scope` . §5.1
- **Tailwind CSS** — Utility-first CSS framework. §13.8
- **TBT** — Total Blocking Time — lab proxy for responsiveness. §14.3
- **Text-level semantics** — `em` , `strong` , `code` , `abbr` , `time` , …. §2.6
- **TLS** — Encryption layer of HTTPS. §1.4
- `<track>` — Captions/subtitles for media. §4.6
- **Transform** — Moves/rotates/scales the painted box. §12.3
- **Transition** — Animates a property change. §12.1
- **TTFB** — Time To First Byte. §14.1
- **URL** — Address: scheme, host, path, query, fragment. §1.2
- `:user-invalid` — Invalid after the user interacted. §5.5
- **User-agent stylesheet** — The browser's default CSS. §13.13
- **Utility-first CSS** — Composing single-purpose classes. §13.8
- **Validation (HTML)** — Checking markup against the spec. §2.10
- **View transition** — Animated morph between DOM states. §12.10
- **Viewport** — Visible area of the page. §10.2
- **Viewport meta tag** — Sets the layout viewport on mobile. §10.2
- **WAAPI** — Web Animations API. §12.6
- **WCAG** — Web Content Accessibility Guidelines (2.2). §3.9
- **Web font** — Font downloaded with the page (`@font-face` , WOFF2). §11.1
- **WebP** — Image format, typically smaller than JPEG/PNG at similar quality. §4.2
- `:where()` — Like `:is()` with zero specificity. §13.2
- `will-change` — Hint that a property will animate. §12.7
- **Writing mode** — Horizontal or vertical text flow. §13.6

- `z-index` — Stacking order of positioned boxes. §7.8

Chapter 14 TL;DR

- **Critical rendering path**: HTML → DOM, CSS → CSSOM, render tree, layout, paint, composite. CSS is **render-blocking**; classic scripts are **parser-blocking** and also wait for pending CSS.
- **Render-blocking fixes** (measured, CSS 800 ms / JS 600 ms): all-blocking FCP 848 ms; `defer` only → 844 ms (CSS was the bottleneck); inline critical CSS + non-blocking stylesheet + `defer` → **40 ms**.
- **Core Web Vitals** (p75 of real visits): **LCP ≤ 2.5 s**, **INP ≤ 200 ms** (replaced FID in March 2024), **CLS ≤ 0.1**. LCP = last largest paint before input; INP = worst interaction's input delay + processing + presentation; CLS = largest session window of unexpected shifts.
- **Measured**: bad page LCP 2648 ms / CLS 0.195 / INP 304 ms → fixed page 1844 ms / 0 / < 16 ms, by putting the hero `<img>` in the HTML with correct `width` / `height` and `fetchpriority="high"`, reserving the banner's space, deferring the script, and yielding in the click handler.
- **Fix lists**: LCP (TTFB, discovery, size, render delay); INP (no long tasks, yield, small DOM); CLS (dimensions, reserved space, nothing inserted above, font metrics).
- **SEO**: title, meta description, canonical (hint, absolute), `lang`, viewport, no accidental `noindex`, `robots.txt` blocks crawling not indexing, semantic headings/links/alt, **JSON-LD** schema.org structured data (eligibility, not a guarantee), keywords meta ignored.
- **Revision**: 47 interview questions (§14.6), cheat sheets for selectors, specificity, box model, flexbox, grid, units and responsive patterns (§14.7), and a 212-term glossary (§14.8).

That's the end of the HTML & CSS Handbook. Next steps: the JavaScript Handbook for behaviour and the DOM (start with its Chapter 1, or jump to Chapter 13 — The DOM and Events if you already know the language), then the React Handbook for component-based UIs. Back to the front page of this book.